SmartGym Adaptive Training System

Measurement and Calibration Guide

ROM math, calibration flow, signal errors, validation, and debugging.

Version 1.0 Prototype

ESP32-S3 firmware | Firebase RTDB | React dashboard

Generated documentation package

SmartGym Adaptive Training System

Measurement and Calibration Guide

ROM math, calibration flow, signal errors, validation, and debugging.

Version 1.0 Prototype

ESP32-S3 firmware | Firebase RTDB | React dashboard

Generated documentation package

SmartGym Measurement and Calibration Guide

Language: English | [Espanol](MEASUREMENT_AND_CALIBRATION_GUIDE.es.md)

This guide explains how SmartGym measures motion, converts raw sensor readings into usable biomechanical values, calibrates user ROM, handles pin load, and detects common error cases. It is written for someone who did not build the project and needs to understand how the system works end to end.

Purpose

The firmware must answer these questions during a workout:

- Where is the machine in its stroke right now
- Is the user moving up or down
- Did a valid repetition just happen
- What physical load was on the machine
- What recommendation should be shown next

To answer those questions, SmartGym combines:

- An analog motion sensor on the machine stroke
- User-specific ROM calibration
- A ROM-based repetition state machine
- A machine pin-load model
- Local session recording
- Firebase synchronization

Big Picture

The measurement pipeline is:

1. Sample the analog motion sensor every 5 ms.
2. Smooth the raw sensor reading with an EMA filter.
3. Convert the filtered reading into a machine position.
4. Normalize that position into percent ROM using the calibrated bottom and top points for the active user and machine.

1. Compute velocity from the change in normalized position over time.
2. Run a repetition state machine on the ROM signal.
3. Record accepted reps, sets, timing, pin load, and session metrics.

4. Save the finished session locally and upload it to Firebase in phases.

Sensor Inputs

Motion Sensor

The main firmware uses an analog motion sensor connected to GPIO 17 on the ESP32-S3 display board. The current firmware assumes one primary stroke signal for the active machine.

The sensor is sampled every 5 ms, which corresponds to about 200 Hz:

```
sample_interval = 0.005 s
sample_rate = 1 / 0.005 = 200 Hz
```

This rate is fast enough to resolve normal gym-machine motion while still being practical on an ESP32-S3 that is also running LVGL and Firebase networking.

RFID

RFID is used to identify the active user. The firmware must know the user before it can load the correct ROM calibration and recommendation.

Pin Load

The firmware currently treats machine pin load as machine state, not user state. In the main firmware, the user updates the current pin load with the screen buttons. The separate ‘weight_detection/‘ folder contains a VL53L0X prototype for automatic weight-stack detection, but that prototype is not the main firmware path yet.

Raw Signal to Position

The analog signal is read by the ESP32-S3 ADC. For a 12-bit ADC, the raw range is:

```
0 to 4095 counts
```

If the full machine stroke is treated as 2000 mm, a simple theoretical resolution estimate is:

```
resolution_mm = 2000 / 4096 = 0.488 mm per count
```

That is a useful engineering estimate for documentation, although the real effective resolution depends on sensor noise, mounting, ADC behavior, and the mechanical linkage.

If the analog voltage is modeled as proportional to stroke position:

```
position_mm = (adc_or_voltage / full_scale) * stroke_length_mm
```

Where:

- ‘position_mm‘ is the estimated position along the stroke
- ‘adc_or_voltage‘ is the measured signal
- ‘full_scale‘ is the sensor full-scale reference
- ‘stroke_length_mm‘ is the machine stroke assumed for conversion

The exact conversion may differ slightly by machine or sensor implementation, but this is the conceptual model used by the project.

Filtering

The signal is smoothed with an exponential moving average, or EMA. The project documentation and firmware behavior use a smoothing factor around:

```
alpha = 0.50
```

The filter form is:

```
filtered[k] = alpha * raw[k] + (1 - alpha) * filtered[k - 1]
```

Why this is needed:

- The raw sensor can jitter due to ADC noise
- Mechanical vibration creates false slope changes
- Velocity is a derivative, so it amplifies noise

Too little filtering causes false rep edges and unstable velocity. Too much filtering causes lag and can flatten short pauses or fast transitions.

ROM Calibration

ROM calibration is user-specific and machine-type-specific.

The system stores a bottom point and a top point for the active user and the active machine type. These points define the useful motion span for that user.

Calibration Inputs

The calibration flow asks the user to:

1. Move to the bottom position and confirm it.
2. Move to the top position and confirm it.
3. Confirm the current pin load.
4. Perform controlled reps.
5. Save the result and recommendation.

ROM Normalization

Once bottom and top are known, the firmware converts the current stroke position into percent ROM:

```
rom_percent = ((x - x_bottom) / (x_top - x_bottom)) * 100
```

Where:

- ‘x‘ is the current filtered position
- ‘x_bottom‘ is the calibrated bottom point
- ‘x_top‘ is the calibrated top point

This normalization is the heart of the system. It lets the firmware compare reps in a user-relative way instead of relying only on absolute sensor values.

Why User-Specific Calibration Matters

Different users do not always use identical mechanical endpoints:

- limb length differs
- preferred setup differs
- seat or attachment position may differ
- some users avoid fully locking out

Without user calibration, the same raw stroke can produce misleading ROM percentages and false judgments about rep quality.

Velocity Calculation

Velocity is computed from the change in position over time:

```
velocity = (x[k] - x[k - 1]) / dt
```

If the project is using normalized ROM directly:

```
velocity_rom = (rom[k] - rom[k - 1]) / dt
```

Where:

- ‘x[k]‘ or ‘rom[k]‘ is the current sample
- ‘x[k - 1]‘ or ‘rom[k - 1]‘ is the previous sample
- ‘dt‘ is the sample interval, about 0.005 s

The important point is that SmartGym uses motion trend and timing, not only a single position threshold. That is what enables rep-phase detection and timing feedback.

Repetition Detection

The firmware uses a ROM-based state machine. The documented states are:

- ‘Bottom‘
- ‘Ascending‘
- ‘Top‘
- ‘Descending‘

Simplified Logic

1. At the bottom, the system waits for a meaningful upward movement.
2. If ROM is rising consistently, the rep enters ‘Ascending‘.
3. If the user reaches the upper target region, the rep enters ‘Top‘.
4. When ROM falls again, the rep enters ‘Descending‘.
5. When the user returns to the lower region, the rep can be counted as complete if all other validity checks pass.

What Makes a Rep Valid

A rep is not accepted just because the signal moved. Typical validity checks include:

- sufficient ROM range
- arrival near the calibrated top point
- reasonable duration
- coherent phase order
- no severe noise or aborted motion

Why Reps Can Be Rejected

A rep may be rejected for reasons such as:

- the user never reached enough ROM
- the top was not reached
- the motion reversed too early
- the signal was noisy
- the movement duration was too short to be trustworthy

This is why a user can move the machine and still not get a counted rep.

Set Completion and Session Recording

When valid reps accumulate, the session recorder stores:

- rep number
- set number
- timestamps
- ROM metrics
- timing metrics
- pin load used for that rep
- rep validity and quality information

This design matters because the machine pin load may change during a workout.

The firmware therefore records the load associated with the rep, not just a

single load for the entire session.

Current Pin Load vs Recommendation

This distinction is essential for understanding SmartGym.

Pin Load

Pin load is the physical selector position on the machine. It belongs to the

machine state and represents what is actually set at that moment.

Recommendation

Recommendation is user-specific advice derived from calibration or prior session data. It does not automatically move the machine pin.

Practical Example

If the machine is physically set to 20 kg and the active user recommendation is 25 kg:

- pin load remains 20 kg
- recommendation is shown as 25 kg
- the session starts at 20 kg unless the user changes the pin

This separation prevents the software from pretending that physical hardware changed when it did not.

Recommendation Generation

The project uses a load-velocity concept for recommendations. The simplified model is:

```
v = a + bL
```

Where:

- ‘v‘ is a representative movement velocity
- ‘L‘ is load
- ‘a‘ and ‘b‘ are fitted constants

With multiple loads and measured velocities, the firmware can estimate how the user performs across effort levels and derive a next recommended load. The exact heuristic may vary by goal, but the core idea is that performance at known loads informs the next load suggestion.

Calibration Outcome

A calibration result usually needs to save:

- calibrated bottom position
- calibrated top position
- quality/confidence
- current or tested load
- recommended next load

- machine type identifier
- user identifier

These values are stored under:

```
calibrations/{uid}/{machineTypeId}
```

Firebase Data Flow

The upload path is intentionally phased because the ESP32-S3 is memory constrained during TLS traffic.

When a session finishes:

1. The firmware creates a local session result.
2. The result is stored in an NVS-backed queue.
3. A sync service uploads the session in phases.

Typical phases include:

- session root
- daily metadata
- daily timeline
- daily summary
- weekly summary
- set details
- rep sets

This design prevents a large single upload from blocking the device or losing data when Wi-Fi is unstable.

Memory Constraints

The firmware must manage two conflicting needs:

- rich UI behavior with LVGL
- TLS-backed Firebase uploads

To reduce risk:

- optional UI trees are freed before heavy uploads
- uploads are chunked when heap is low
- an NVS queue stores sessions for retry
- large rep-set payloads are split into smaller writes

Why This Matters

If memory is low and a large payload is sent anyway, the device can freeze, stall, or fail the upload. This is one of the most important implementation realities for anyone reproducing the project.

Error Sources and Failure Modes

1. Mechanical Alignment Error

The motion sensor or ToF holder may be mounted at a slightly wrong angle or position. This causes:

- compressed stroke range
- inconsistent top or bottom detection
- bad ToF target readings

2. ADC Noise

Electrical noise can produce jitter in the analog signal. Symptoms include:

- unstable graph lines
- false phase transitions
- inconsistent velocity

3. Bad Calibration

If bottom or top are saved incorrectly:

- ROM percentages become misleading
- good reps may be rejected
- top or bottom may never be reached in software

4. User Technique Variability

Different body positioning, tempo, partial reps, or bouncing can create signal patterns that differ from the ideal expected motion.

5. Network or Cloud Failure

The workout may finish correctly while the upload is delayed. This is why local queueing is required.

6. Memory Pressure

The ESP32-S3 may have enough CPU to continue running, but not enough safe heap to perform large Firebase writes. This shows up as delayed uploads, chunking, or

backoff behavior.

How to Validate a New Installation

If someone new is reproducing the project, this is the recommended validation sequence.

Hardware Validation

1. Confirm the motion sensor is physically secure and moves through the full machine stroke.
1. Confirm the display boots and touch input works.
2. Confirm RFID reads a test card.
3. Confirm the ToF holder is mounted consistently if the prototype is used.

Signal Validation

1. Watch the live graph while manually moving the machine slowly.
2. Confirm the raw stroke direction matches the actual machine direction.
3. Confirm the graph is not saturating too early.
4. Confirm the signal reaches both low and high regions cleanly.

Calibration Validation

1. Save bottom.
2. Save top.
3. Move the machine again.
4. Confirm ROM rises from near 0\% to near 100\%.
5. Perform a few smooth reps and confirm they are accepted.

Session Validation

1. Scan RFID.
2. Wait for user sync to complete.
3. Start a session.
4. Perform reps.
5. Finish the session.
6. Confirm the summary appears.
7. Confirm the session enters the Firebase queue and eventually uploads.

How to Debug Wrong ROM

If ROM behaves incorrectly:

1. Check whether bottom and top were saved in the correct order.

2. Check whether the machine stroke actually reaches the expected endpoints.
3. Check whether the sensor is mounted too loosely.
4. Check for severe noise in the graph.
5. Re-run calibration with slow, full-range motion.

How to Debug Wrong Pin Load

If pin load is wrong:

1. Verify whether the main firmware is in manual pin-load mode.
2. Confirm the user updated the on-screen pin load buttons correctly.
3. If using the prototype ToF module, recalibrate its machine-specific distance ranges.
1. Check alignment of the ToF target and holder.

How to Explain the Project to a New Teammate

A good short explanation is:

```
SmartGym samples machine motion every 5 ms, smooths the signal, converts it
into normalized ROM, detects rep phases from that ROM, records reps and sets
with the current machine pin load, and then uploads the finished session to
Firebase using a memory-safe queued pipeline.
```

If the new teammate understands that sentence, they have the mental model needed to navigate the rest of the system.

Recommended Companion Documents

- [Project README](../README.md)
- [System Architecture](SYSTEM_ARCHITECTURE.md)
- [Developer Guide](DEVELOPER_GUIDE.md)
- [Firebase Guide](FIREBASE_GUIDE.md)
- [Troubleshooting Guide](TROUBLESHOOTING.md)
- [Weight Detection Prototype](../weight_detection/README.md)